Evaluating the capabilities and hardware limitations of Edge AI VS Cloud AI

UNIVERSITY OF
LINCOLN

Matthew Cornwell
27208357

27208357@lincoln.ac.uk

School of Engineering and Physical Sciences

College of Health and Science

University of Lincoln

Submitted in partial fulfilment of the requirements for the
Degree of BSc(Hons) Computer Science

Supervisor Dr. Bashir Al-Diri

May 2026

Acknowledgements

Firstly, I want to thank my friend Marsh for putting up with me, and George for his sofa (and also putting up with me).

Thank you to all my friends, family and teachers past, present and future, wouldn't be here without any of you.

Abstract

There has been a major global shift towards AI, resulting in interconnected ever present challenges of privacy and environmental sustainability in the modern day, specifically with privately owned cloud data centres being the powerhouse behind AI growth. This project aims to solidify edge AI as a potential solution to these problems.

Despite the surge in edge AI research, there is a lack of empirical unified comparisons between NPUs, GPUs and Cloud that measure inference speed, energy per token and response quality. The core goal of this project is to close this gap by building a reproducible cross platform benchmarking artefact across the RTX 4070 GPU, RK3588 NPU and a cloud baseline in the form of Gemini 2.5 Flash. It will do so by utilising the Qwen3-4B model, evaluated against the industry standard IFEval dataset with a custom retrieval augmented generation pipeline running against the Simple English Wikipedia dataset.

The strongest finding of this project is that the RK3588 NPU proved to be 3.6x to 5.5x more energy efficient per token than the RTX 4070 GPU, with the quality gap on IFEval pass rates being a minor ~5% across all 3 backends. Edge AI, particularly NPUs, are currently a highly practical, energy conserving, private alternative to cloud based AI for inference and widely underused.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation & background

The main rationale behind the shifting research focus to edge AI addresses two primary, interconnected concerns: privacy and sustainability. Although cloud based models currently offer a powerful jack-of-all trades output, they are distinctly lacking on both of these fronts. If edge AI is able to compete on energy efficiency and offline use cases, then global sustainability and data privacy standards would be much easier to uphold in personal, academic, political and business ventures.

As far as privacy is concerned, Cloud-based AI assistants raise major concerns that can arguably only be addressed by offline edge AI alternatives. In 2023 a landmark lawsuit between the Federal Trade Commission and Amazon, with the FTC alleging that Amazon was retaining child voice recordings indefinitely (FTC, 2023) (note that Amazon settled for $25M and denied all charges). This is but one example in an array of many that shows these large monopolies do not care about civilian rights, and with AI scaling as fast as it is and being integrated into every possible device, AI privacy should in my opinion be of utmost concern to the public to avoid the acceleration of Orwellian states globally.

In terms of energy usage, a recent study shows "U.S. data centre energy report predicts that AI servers' electricity consumption in the U.S. alone will surpass 150 – 300 TWh in 2028" (Li et al., 2025), with the US using "94.21 quads" of energy a year (U.S. Energy Information Administration, 2024), which comes to roughly 1% of their total yearly energy usage. To put this in perspective, 300 TWh of energy

is roughly the total energy consumption of a country like Italy, or the combined electricity usage of tens of millions of homes.

In general as with most sustainability efforts, the end users are left with the responsibility of helping the planet, not the conglomerates. This has led the conversation down the route of managing inference, instead of training. The US government is hardest on scaling up massive datacenters such as their $250 billion Stargate project, so in terms of what the average every day AI user can do to stop that, the only option I personally see is the switch to offline, efficient local inference.

Despite these motivations, AI in general is still mainly led by power hungry progress driven executives, leaving edge AI research behind somewhat. This is the knowledge gap I attempted to help close in my own way with this project. Edge AI has a few flaws, namely slower inference, lower quality output due to necessary quantization, and limited context windows. I will thoroughly explore this and conclude with data analysis on the current state of edge AI in 2026.

1.2 Aims & objectives

The overarching goal of this project has been to create a reproducible, cross platform benchmarking artefact that thoroughly evaluates Large Language Model capabilities and inference across multiple different hardware platforms, with focus placed on energy efficiency, privacy and output quality under Edge AI specific constraints.

The following SMART objectives guided the entire process:

Objective 1: Deploy a standardised benchmarking pipeline across three main platforms: An RTX 4070 GPU, A Rockchip RK3588 NPU, and Gemini API (as a cloud baseline), by the end of March 2026

Objective 2: Evaluate energy per token across all platforms using Joules per token, derived from platform appropriate energy measuring metrics, and across a large enough dataset to achieve a solid average, taking inspiration from other efficiency papers in the field (Oviedo et al., 2025)

Objective 3: Assess the instruction following ability on all platforms using the industry standard IFEval dataset (Zhou et al., 2023), using heuristic based verification methodology, producing a detailed cross platform prompt response quality comparison

Objective 4: Implement and benchmark a Retrieval Augmented Generation (RAG) pipeline utilising FAISS vector indexing on the entirety of Simple English Wikipedia (HuggingFace, 2025), measuring the impact across prefill latency, energy cost and response quality relative to the non RAG baseline.

Objective 5: Create a reproducible and demo-able artefact in a Python Virtual Environment that enables further cross platform development, which can also be recreated with ease by independent future researchers

1.3 Dissertation structure

This dissertation proceeds with the following structure. Chapter 2 functions as a literature review, covering the environmental concerns surrounding AI inference, relevant hardware architectures, quantization and compression of models, and edge device challenges. Chapter 3 defines the overall requirements for the project and benchmarking artefact. Chapter 4 covers the design of the system, followed by the rationale of the measurement methodology. Chapter 5 details the implementation of the system, including cross platform deployment and roadblocks hit with the burgeoning NPU technology of the RK3588. Chapter 6 presents and opens a discourse about the results across all 3 platforms. Chapter 7 concludes with an overall summary of the research, limitations and possible future improvements.

Chapter 2

Literature Review

2.1 Environmental impact of AI inference

While initial training of models has a high carbon cost, the real impact environmentally is the cumulative use of the model post training. Previous studies have focused on the efficiency of mobile processors or simulations (Chen et al., 2025) in the past, revealing a gap in the research for efficiency and privacy comparisons of Single Board Computers. The SBC chosen for this project is the Orange Pi 5 Plus with the Rockchip RK3588's NPU and 16GB of RAM.

Some researchers argue that "if the whole ML field adopts best practices, that by 2030 total carbon emissions from training will decline" (Patterson et al., 2022), which I personally believe is an optimistic view, as the ML field continually proves itself to be more profit than efficiency minded. The landmark paper by Strubell, Ganesh and McCallum, 2019 mainly focused on this sort of training for Deep Learning which relates to how current models are trained, comes to the same conclusion: "Researchers should prioritize computationally efficient hardware and algorithms".

Microsoft, who pledged carbon negativity by 2030, saw its emissions rise by 30% since they made that commitment, mainly driven by AI data centre construction as stated in the following "Microsoft said its emissions grew by 29% since 2020 due to the construction of more datacenters that are "designed and optimized to support AI workloads." (Kerr, 2024). With Google following suit too according to Kerr and official statements from Google, the overall sustainability movement that grew throughout the 2010s has come crumbling down, in what I personally believe is a

race to the most powerful AI, with nature and conservation taking a backseat in the conversation.

This is not a reputational failure by these companies, it is a global economic structural failure. The amount of power required to train the largest flagship models has been growing at more than 2x per year, and it is only trending upwards. These companies have little commercial reason to slow this down, when their capability increases (regardless of efficiency), their share prices directly follow the same trend. This constant churn also creates the problem of wasted training, if a company trains an X.1 model and two weeks later releases an X.2 model that's 2% better, all of the energy and training for the first model is wasted. This leads to an industry which treats energy consumption (and thus our planet) as a roadblock, not a viable constraint, precisely the dynamic which requires, in my opinion, a shift to edge focused AI for inference at the very least.

There is an identifiable split in energy consumption into two distinct stages, the decode and the prefill time of the models. This demands the ability to precisely measure the energy differences at each moment by code. To do so accurately, the project uses pynvml for software based power monitoring, enabling full data analysis of all energy usage at each step of the pipeline There is also a priority on Green AI which puts "efficiency as a primary evaluation criterion alongside accuracy." (Schwartz et al., 2020). The WEF even states "By strategically harnessing AI to enhance our renewable energy landscape, the future of AI holds the promise of not only becoming green in its own operations but also aid in building a more sustainable world for generations to come." (Ammanath, 2024), which led to the decision of taking "energy per successful task" as a critical metric, which will be graded using aforementioned heuristic validation covered in depth later.

2.2 Hardware architectures

The hardware ecosystem or spectrum for running AI is frequently misunderstood as a linear hierarchy, when in fact there are two main polarities: compute power vs energy efficiency and specialisation. CPUs work as highly versatile generalists for most AI,

but lack powerful enough parallelism for efficient matrix multiplication, where GPUs specialise. On the other hand, said GPUs range from consumer level hardware such as the 4070 utilised in this project, to immense clusters driving cloud data centers. These GPUs have been the leading force behind the AI boom, causing scarcity in the consumer market due to their extremely efficient matrix multiplication and high Tera Operations Per Second (TOPS) throughput. However this computational excellence comes at the cost of high energy consumption.

Neural Processing Units (NPUs) represent a divergent architectural philosophy, spearheaded in recent years by Chinese companies: "China's AI labs have shown that you do not need the best model to win the market; you need the cheapest one that is good enough" (AmericanEnterpriseInstitute, 2026), mainly due to America controlling the GPU market. Research by Nguyen et al. (2025) confirms this, declaring the Pi 5 as a "mid-tier inference platform" and finding RK3588 systems show "the strongest performance across the board".

To fully explore the specific hardware relevant to this project, I will now discuss the Rockchip RK3588, which features a dedicated three core NPU capable of 6 TOPS at INT8 precision, with approximately 34 GB/s of LPDDR5 memory bandwidth. These specifications are ideal for local Large Language Model execution, as "the performances of swapping and recomputation depend on the bandwidth between CPU RAM and GPU memory and the computation power of the GPU" (Kwon et al., 2023), the 34 GB/s of memory bandwidth specifically being paramount in the decision to use the RK3588. In comparison, the consumer level RTX 4070 has a bandwidth of roughly 192 GB/s, 5.6x that of the RK3588. This disparity in bandwidth can assist in predicting the baseline for expected token output, and illustrates how the NPU works for power efficiency over raw throughput.

Finally, to contextualise both the RTX 4070 and RK3588 in the modern day environment, this study will utilise Cloud AI (specifically gemini-2.5-flash) as a quality baseline. While these Cloud AIs currently set the ceiling for capabilities on benchmarks such as the chosen IFEval, they have a few distinct drawbacks. Significant latency on the round trip over the network being one of them, although it can be

argued that this is minor and "worth the wait" for a higher quality response. Personally the main drawbacks I see link back to the main themes of this project, privacy and energy consumption. Google openly admits that they own whatever you prompt or receive as a response, and are entirely opaque when it comes to discussing energy usage. This means that the project will use Gemini as specifically a quality anchor, not a privacy or energy anchor, as any comparison there is borderline moot - there is no reliable privacy or energy information available.

2.3 Quantization and model compression

Edge AI deployment on LLMs requires significant compression of the models through quantization, a “practical technique for making large language models easier to deploy” (Kurt, 2026). Besides solely storage reduction, quantization helps navigate memory bandwidth which is the largest bottleneck of edge AI. Kurt concludes that the Q4_K_M format is “tuned for speed / quality” so it forms the basis of the project’s tests, although other methods will be tested and evaluated throughout. Kurt also states that quantization replaces FP16 weights with lower precision integers such as INT8 or INT4, reducing the overall memory bandwidth requirements.

There are many different techniques and methods for quantization, not solely Q4_K_M. Dettmers et al., 2022 developed "a two-part quantization procedure, LLM.int8().", to deal with the fact that modern transformer language models "dominate attention and transformer predictive performance", aiming to preserve accuracy through outlier-aware decomposition. Frantar et al., 2022’s GPTQ took a different approach, utilising "a new one-shot weight quantization method based on approximate second-order information, that is both highly- accurate and highly-efficient". Lin et al., 2024 took a different angle, noting that "protecting only 1% of salient weights can greatly reduce quantization error.", leading them to "observe the activation, not weights". Each of these differing approaches represents a trade off between calibration, hardware compatibility, and accuracy preservation

Research by Shi and Ding, 2025 warns that using a model with more aggressively compressed parameters can lead to hallucinations and forced reprompts. They also

argue that if a quantized model requires multiple prompts to reach a decisive answer then using a larger, more accurate model would be more energy efficient. To verify this threshold and fully explore the current landscape of edge AI, this project will test several of the best Small Language Models (SLMs). Subramanian, Elango and Gungor, 2025 survey on SLMs informs our basis for chosen models: both Llama-3.1-8b and Qwen3-4B are tested at Q4_K_M compression, although a custom INT8 algorithm is utilised and will be covered later for the Orange Pi 5 Plus model.

2.4 Edge inference, latency, and RAG

Edge inference presents a few unique challenges, namely response latency and lack of information due to the compression / quantization. The latency is important as a high latency increases frustration in the user, meaning Time-To-First-Token (TTFT) is a highly valuable metric. Although implementing a working voice assistant pipeline is outside the scope of this project, user satisfaction requires that “voice assistants have a latency of no more than 5 seconds” according to Sogemeier et al., 2024, meaning that if a viable TTFT is reached then local AI models are entirely practical for real time conversation, a useful metric to keep in mind. A meta analysis from Brysbaert, 2019 of 190 studies states that ~18,000 participants find silent reading averages at 238 words per minute for non fiction, and a separate sub analysis shows reading aloud at 183 wpm. This means that roughly a 5 TPS conversational floor aligns well with human cognitive capacity, meaning we will compare our results against this threshold.

End to end latency splits comfortably into two main bottlenecks: Prefill TPS and Decode TPS. Prefill is the total compute time of processing the initial prompt and is compute bound, whereas decode is the following sequential token generation and is memory bandwidth bound. Prefill is compute bound as it can "efficiently use the parallelism inherent in GPUs" Kwon et al., 2023, as "the prompt phase can be parallelized using matrix multiplication operations". For decode on the other hand, "the autoregressive generation phase generates the remaining new tokens sequentially.", meaning that the bottleneck is due to the fact that "different iterations cannot be

parallelized due to the (sequential) data dependency and often uses matrix-vector multiplication".

The knowledge gap surrounding quantized local AI leads to Retrieval Augmented Generation (RAG) being useful for increasing the validity of a response while simultaneously keeping energy costs down and retaining data privacy. An implementation of MiniRAG (Fan et al., 2025) will be tested in this project, as it introduces a "graph indexing mechanism" that improves response speed and efficiency.

2.5 Research gap & summary

The literature covered here addresses multiple topics leading to a gap which will be summarised here. First, data centre level efficiency is well covered through many angles, and per task energy methodology is covered at this scale too. Secondly, edge AI inference and SBC throughput are increasingly being studied over the last few years, though typically without energy measurements and privacy specifically in mind. Third, retrieval augmented generation on SBCs is slowly emerging onto the field. What hasn't yet been covered is an empirical study that measures inference speed, per token energy, and overall quality of the responses between two main local platforms - in this case an RTX 4070 and an RK3588, compared against a cloud baseline on an identical set of prompts.

This project will aim to address and shrink this gap. The contribution isn't hardware or model based, but unified comparison from a single source of truth script, delivering this across 3 platforms and graded against a solid quality benchmark, with different models and RAG settings covered to find the current usability, privacy and energy efficiency of current and future potential edge AI devices. Chapter 3 will cover the requirements to follow this aim effectively, and Chapter 4 details the methodology that will allow this to take place.

Chapter 3

Requirements Analysis

This chapter formalises all of the requirements that the artefact must include to address the research gap identified in the previous. This requires the artefact to have a unified comparison of inference speed, energy per token and quality results across the GPU, NPU and cloud platforms on the same prompt set. As this is a research project and not a consumer oriented product, all the requirements will be derived specifically from what must be true of the artefact for the results to be meaningful, not from other methods like user interviews. The SMART objectives from Section 1.2 will guide this process throughout.

Section 3.1 will cover functional requirements, 3.2 the non functional, 3.3 the operating environment, and 3.4 the bridge that leads into the methodology in Chapter 4.

3.1 Functional Requirements

These functional requirements describe what the finished artefact requires for the final results to be a fair and valid comparison across the 3 platforms. Each of them is derived from the SMART objectives.

FR1: Cross platform inference execution. The artefact must be able to execute a defined IFEval prompt set against the three backend modes: llama_cpp for the GPU, rkllama for the RK3588 NPU, and the Gemini API for the cloud baseline comparison. All of this has to be abstracted through a common BaseBackend interface to ensure that the results and inference are backend agnostic. *Objective 1*

FR2: Metric capture for each prompt. For every prompt, benchmark or mode, the artefact has to record: TTFT, total time, prefill TPS, decode TPS, total energy (J), average and peak power (W), energy per output token and the full prompt and response text. *Objectives 1 and 2*

FR3: RAG Mode. The artefact must support a Retrieval Augmented Generation (RAG) mode that retrieves the top chunks from an FAISS index of the entirety of Simple English Wikipedia, augment the prompt before generation, and tag the mode in the output CSV as either "raw" or "rag". *Derives from Objective 4*

FR4: Resume after interruption. The artefact must be able to scan an existing results CSV before starting a run, and skip any prompt with results already generated in the CSV. This will allow long unattended runs without oversight on slow platforms to resume after interruption, maintaining data consistency. *Derives from Objective 1 for edge platforms*

FR5: Response grading. An impartial, separate grading script must be implemented to evaluate each response against the prompt using heuristic verifiers, producing strict and loose pass labels alongside prompt specific outcomes for later audit and analysis *Derives from Objective 5*

FR6: Single CSV schema. All platforms must be able to emit results to a single CSV with a stable schema across all platforms, enabling consistent cross platform analysis from a single source of truth *Derives from Objective 5*

3.2 Non Functional Requirements

These non functional requirements describe specifically how the artefact must operate, rather than what it must do, enforcing strict reproducibility, reliable measurements, resilience and auditability, derived from knowledge gained from the Literature Review.

NFR1: Reproducibility. Each run of the artefact must be repeatable from a single command-line invocation utilising flags when calling the Python script. The CSV must capture the model, platform string, energy monitoring method, run mode, the

run index, and timestamp for each run, making the conditions of each run entirely reproducible.

NFR2: Portability. The script must run on Windows in a Python Virtual Environment, with compiled CUDA libraries for the RTX 4070 path, and an Orange Pi 5 Plus specific version of Ubuntu Server "ubuntu-22.04-preinstalled-server-arm64-orangepi-5-plus.img.xz", with backend selection primarily driven by CLI flags from the main script running on the Windows host PC. The Gemini path will also run through this Windows driven Python Venv host.

NFR3: Measurement validity. Energy measurement is required to use platform appropriate counters with high accuracy. The host script uses pynvml on the GPU and wall clock based sysfs telemetry calculations on the RK3588. The cloud path has no energy claim due to Google's secrecy, a limitation following on from discussion in the literature review. Per platform method limitations are documented in Section 4.2

NFR4: Resilience. Long runs must survive chance backend failures. Prompt specific errors must be logged into the CSV notes field, but must not crash the program and allow future prompts to be ran in-spite of the failure. The resume capability noted in FR4 complements this by allowing a restart and resume on failed runs

NFR5: Auditability. Full prompt and response text strings must be stored in a respective CSV row, enabling manual human verification alongside the IFEval grading outcomes, without having to re run inference and waste valuable time and energy.

3.3 Hardware / Software Environment

This section covers the hardware and software environments in which the artefact was built and evaluated, and discloses a couple known quirks of the environment.

Platform	Hardware	OS / Kernel	Runtime	Energy method	Models
Edge GPU	Lenovo Legion Slim 5 16IRH8, RTX 4070 Laptop (8 GB VRAM), Intel Core i7-13700H, 16 GB RAM	Windows 11	llama-cpp-python (CUDA)	Tapo P110 Smart Plug with pynvml @ 10hz as a backup	Qwen3-4B Q4_K_M (GGUF)
Edge NPU	Orange Pi 5 Plus, Rockchip RK3588 (3-core NPU @ 6 TOPS INT8), 16 GB LPDDR5	Ubuntu 22.04 Server LTS, 5.10.0-1012-rockchip kernel	rkllama (Flask wrapper of RKLLM 1.2.3)	Tapo P110 Smart Plug	Qwen3-4B (RKLLM-converted, Q8_K_M)
Cloud	Google Gemini API Free Tier	N/A	google-generativeai package	N/A	gemini-2.5-flash

The Orange Pi 5 Plus device used in this project runs rknpu driver version 0.9.6 on kernel 5.10.0. The RKLLM runtime requires driver version 0.9.7, which is only available on the newer 6.1+ images supplied by Rockchip's newer (unstable) firmware bundles. Upgrading the kernel would require a full system reflash, which at the point of discovery was outside of the scope for this project. This driver mismatch

may be a slight performance hit on the NPU, and a known constraint for potential reproducibility, hence the mention here. This implies that the results in Section 6 should be treated as the floor for potential RK3588 capabilities, future work discussed in Section 7.4 includes repeating this benchmark with newer firmware and matching drivers.

3.4 Evaluation Approach

Each SMART objective will be met by the methodology in this section, fully covered in the results reported in Section 6. Objective 1 of a standardised benchmarking pipeline is satisfied by the end to end execution requirement laid out here. Objective 2 is evaluated based on the energy measurement and consistency / reproducibility requirements using platform specific measurements. Objective 3 will be evaluated by the IFEval pass rates with n=100 prompts. Objective 4 (RAG) is evaluated through the testing of both raw and RAG methods outlined above. Objective 5 is evaluated through the singular host script requirement running all the models and benchmarks. Detailed methodology on how this will be done follows in Section 4.

Chapter 4

Design & Methodology

4.1 Project management

For this project I worked solo, using a self imposed iterative design model. Sprints took place between other academic and internship work throughout the year, aligned with dissertation specific milestones (Proposal, Interim report and Final Submission). I used personal notes in markdown on Joplin and personal backups on secondary storage for version control.

Sprint 1 (October to December 2025): I did the literature review, initial development of the host script on the GPU with llama.cpp, and my interim report

Sprint 2 (January to March 2026): I completed the RAG implementation, the IFEval integration, and started building out the NPU deployment

Sprint 3 (April 2026): RK3588 fine tuning and debugging, rkllama integration into the host script, driver mismatch debugging

Sprint 4 (May 2026): Final benchmark runs, dissertation report write up

Time management was a roadblock for regular supervisor meetings due to a hectic year and personal circumstances (internship, bereavement), but I found that if I needed help with anything, Dr. Bashir Al-Diri was quick to respond.

4.2 Risk analysis

Early on I identified multiple overarching risks with this project. The first was acquiring and setting up the RK3588 chip, which was a big unknown until just months ago. The validity of the energy measurement was also in question, and, as discussed later, I had to fall back to software instead of hardware monitoring for it. These and more are covered in the risks and mitigations table below.

Risk 1 and Risk 4 were the dominant risks that caused issues in this project. Risk 1 led to having to spend a lot more time than expected getting the RK3588 running, and Risk 4 meant I had to drop the initially planned 4 platforms to 3. Finding out about these risks was iterative during development, but enough time was allocated overall to mitigate these effectively and still draw viable conclusions from the project.

4.3 Software development methodology

The methodology used for this project was an iterative model, with no formal software development lifecycle implemented as methodologies like Scrum and Waterfall are designed for coordination amongst teams, not solo developer research artefacts. The sprints I undertook aligned with dissertation and other academic milestones, not arbitrary cycles following the 40 hour work week pattern. They were defined by deliverable objectives such as the interim report, NPU setup and the final writeup as discussed in 4.1. All three final backend systems (llama_cpp, rkllama and gemini) were tested end to end before adding the next one, allowing the overall backend script to develop alongside each model.

Markdown notes were utilised in Joplin alongside local version control and iteration using secondary storage. I utilised CSVs as the single source of truth for output data, enabling comparison from all the initial runs in development to the final 100 prompt run without rerunning inference on old datasets or versions. There are some limitations in the methodology, mainly the lack of a GUI and the lack of automated unit tests. I chose this to keep the project scope aimed correctly, but it's likely this would not scale to production without further development.

ID	Risk	Likelihood	Impact	Mitigation	Outcome
R1	NPU runtime maturity (rkllama is relatively new)	High	High	Very simple Ubuntu image specifically for the exact model, acknowledge minor problems (driver mismatch)	Driver mismatch happened, mitigated in relevant sections through discussion
R2	Smart plug energy validation fails	Medium	Medium	Software energy counter as a backup method	Smart plugs didn't work on uni halls' Wi-Fi (discussed further later), software counters as primary measurement. Eventually fixed this using a mobile hotspot, also discussed later
R3	Cloud API rate limits	Low	Medium	Free tier Gemini used and tested	£0.13 was spent on paid tier usage
R4	Scope creep with platforms or models tested	Medium	High	Originally planned 4 platforms (Raspberry Pi 5 CPU)	Had to drop the base Pi 5 from the scope
R5	Timeline mismatch	Medium	High	Outline of dissertation planned early, data independent chapters drafted prior to benchmarks	Partially materialised due to constraints outside of my control, recovered through efficient time management

Table 4.1: Risk Analysis and Mitigations

4.4 System architecture

The system is defined and ran from a singular host script which drives the whole model pipeline, regardless of the targeted platform (*NFR1, NFR2, FR1*). There are three layers:

1) The prompt loader and RAG retrieval

2) Backend abstraction for each of the models

3) The energy monitor and CSV writer

All of this is controlled based on the flags passed in the CLI when running the program, using one BaseBackend() class changeable with –backend llama_cpp | rkllama | gemini, keeping the output and usability consistent across all backends, and allowing the use of the same prompts, energy measuring and data schema. This is done purposefully to allow for an accurate benchmark with consistent overhead, instead of choosing to run the script on each platform separately.

The BaseBackend() class defines two methods: generate(prompt, max_tokens) -> (response, metrics) and count_tokens(text) -> int.

All 3 platforms inherit from this BaseBackend class. LlamaCppBackend utilises llama-cpp-python with CUDA, with direct access to evaluation metrics from llama.cpp for native TPS reporting. RkllamaBackend uses a HTTP client to access rkllama_server running on the Orange Pi 5 Plus over ethernet, with rkllama_server being a Flask wrapper around RKLLM 1.2.3. The only connection between this backend and the host script on windows is a simple API call to flash. Finally, the GeminiBackend uses the new google-genai SDK to call gemini-2.5-flash, with streaming enabled and thinking turned off to match the GPU and NPU. This abstraction allows us to meet FR4 and FR6 as the calls are all inherited from the same base class.

The energy monitor runs at a default of 10hz for polling, taking the power draw data via either the Tapo P110 over a hotspotted local network, or pynvml for the GPU specific data (without laptop overhead). The monitor starts before generate() is called, stops afterwards, and gets the total Joules used during the period. As the

energy monitor runs in parallel, it doesn't affect or block generation regardless of the platform being benchmarked. See Figure 4.1 for a flowchart of the architecture.

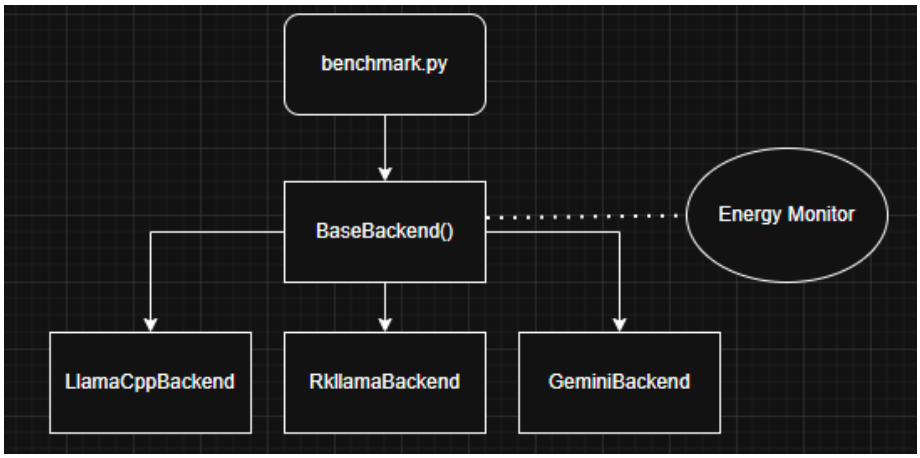

Figure 4.1: System architecture flowchart

4.5 Measurement methodology

Following are the metrics used for the measurements throughout the project, which I will discuss in more depth beyond their initial definitions in the next paragraphs:

TTFT: Time to first token in seconds, the amount of time elapsed between the request and first token received back.

Prefill TPS: The amount of tokens it takes for the model to process and understand the request.

Decode TPS: The speed at which the model generates new sequential tokens after the initial prefill.

Total energy (J): The total energy used during the entire inference process from beginning to end.

Energy per token (J/token): The crucial efficiency metric calculating the total energy used per token generated.

Average and peak power (W): The mean and max of the energy used throughout the generation.

TTFT and Prefill/Decode TPS are crucial metrics for this project. TTFT is recorded from the wall-clock seconds from the beginning of the generate method to the first token output. It is also necessary that prefill and decode are measured separately, as they are both bottlenecked by different things as mentioned earlier. The prefill is compute bound, whereas decode is memory bandwidth bound (Kwon et al., 2023). Prefill TPS is calculated with input_tokens / TTFT, whereas decode TPS is calculated with output_tokens / (total_time - TTFT). This includes ever so slight overhead due to scheduling with the script, but this is conventional for this kind of benchmark.

The primary measurement for energy comes from the TP-Link Tapo P110 smart plug at the wall socket, utilising the local tapo python library at 1Hz, using trapezoidal energy measurement as is industry standard to get the total Joules per prompt. 1 Hz is the ideal energy monitoring sample, as polling any faster would just give repeated data. Using a wall socket for measurement allows us to capture the entire overhead of each system independently, rather than relying on the fallback pynvml for software based readings. Software counterparts usually measure a singular aspect of a device instead of the full picture, but as far as sustainability goes the total cost is most important. The P110 specifies roughly 1% accuracy, well within the boundary of allowing useful data. The TapoEnergyMonitor class runs in a separate thread utilising an asyncio event loop, with a start and stop function called for each generate method call.

We also get measurements of specifically just the GPU's power draw using nvmlDeviceGetPowerUsage() polled at 10hz, integrated the same as the tapo plug. This will allow comparison of the GPU without the overhead of running the entire laptop. Technically, tapo energy - pynvml energy should equal to the system idle baseline, which will be confirmed in the results and discussion in Section 6. The NPU has no comparable software counter, but this is a known limitation and unavoidable.

The R2 risk mitigation discussed in 4.2 actually worked in the end, allowing the utilisation of a mobile hotspot connected to both the laptop running the host script and the smart plug, allowing energy monitoring over that network which would not

have been possible on uni accommodation's obfuscated network. Both full benchmark runs completed the Tapo measurement successfully, with the caveat of sample integrity being noted if the run had fewer than 5 valid energy samples.

One downside of this measurement technique (or just cloud AI in general) is that the Gemini runs cannot be measured. The host script power draw here is negligible, as we're really testing the cost of inference, and Google keeps all that data under lock and key. This means that the Gemini integration functions as a baseline comparison.

4.6 Model and quantization choices

After a couple tests and some research, Qwen3-4B was chosen as the primary model to test across both edge platforms, with gemini-flash-2.5 being a similar powered benchmark against it. The 4B model was chosen as it is the largest class that comfortably fits inside the RK3588's 16GB of memory with adequate headroom for the OS and cache. An 8B model was also briefly tested, but decided against to keep the comparison fairer. The Qwen3 family of models benchmarked at or near State of the Art among its competitors according to Subramanian, Elango and Gungor, 2025. Availability was also a factor in this decision, as both the .gguf format for llama.cpp and the .rkllm format for the NPU formats are readily available, allowing for fair comparison and discourse. Self conversion of models to the .rkllm format is possible, however, outside the scope of this project. Llama-3.1 models were also tested on the GPU, however for NPU access it requires the signing of meta's terms and services tied to a HuggingFace account, defeating the aforementioned privacy angle of this project. Qwen3 on the other-hand is Apache 2.0 licensed, meaning it is fully open source and usable offline with no tracking or oversight. Testing other models was initially planned but cut due to time constraints, discussed in Section 7.1.

For quantization on the GPU model, Q4_K_M was utilised, as it is widely considered the default for edge devices, has been largely benchmarked in similar studies, and is tuned for a balance between speed and quality - all of which makes it a solid choice for comparison. For the NPU, Q8_0 has been chosen as the closest equivalent to

llama.cpp's solid Q4 benchmark. This is due to the fact that the NPU's matrix units are optimised for INT8. Although running Q4 on the NPU is supported, preliminary benchmarking and discourse suggests there is a significant accuracy cost. This asymmetry is a known limitation of the study, as the NPU is in theory running a less aggressively compressed model. This actually understates the NPU's efficiency as it is running a more powerful model than the GPU Q4 counterpart, which will be discussed further in Section 6.

Other parameters such as temperature, top_p, top_k are fixed across all backends to make the comparison meaningful. The max token output is also capped at 1024 by default to bound the total runtime of the software. The vast majority of prompts self complete during this time, whereas with 256 as the max a lot of the prompts were truncated early. The seed cannot be controlled as RKLLM doesn't expose one, but this is mitigated by the large sample size of n=100 prompts.

4.7 RAG implementation design

RAG addresses two main edge AI weaknesses identified in Section 2.4: Quantized models lose factual recall ability in long prompts / responses, and edge devices have a naturally limited context window. The question being explored here is whether or not RAG meaningfully changes any of the measurables of this study, such as energy, latency and quality of edge inference responses, and whether or not the presumably longer prefill cost is worth the trade off for the expected output quality improvement. This satisfies FR3 about RAG.

The corpus used for the RAG pipeline is Simple English Wikipedia, chosen for its broad coverage and shorter articles than the full Wikipedia dataset, reducing irrelevant context and processing. I chose to embed it with the sentence-transformers/all-MiniLM-L6-vs, a small 22M parameter model which is fast on CPU and widely benchmarked (Reimers and Gurevych, 2019). The project indexes this with FAISS IndexFlatIP with an exact inner product search, a cosine similarity search with no approximation, as due to the relatively small roughly 220k articles, approximate in-

dexing would offer little speed advantage. The top-k for the RAG is 2, and the chunk size is ~800KB with 464 total chunks.

Retrieved chunks are then prepended to the prompt, with a brief system instruction of "Use context to answer. If irrelevant, ignore.". For this implementation I took inspiration from MiniRAG, but this implementation is much simpler with no graph indexing or multi-hop retrieval. This is noted in Section 7.4 related to future work.

4.8 Evaluation protocol and dataset

The IFEval dataset was chosen for this project as it specifically tests instruction following by utilising verifiable constraints such as word counts, formatting, forbidden words, etc. as opposed to a subjective quality metric from humans or another LLM. This matches the NFR1 reproducibility requirement, as the same prompt and response will always produce the same grade. The n=100 prompts sampled for the final benchmark come from the full 500+ prompt set, covering all the different instruction types to achieve the widest benchmark in as many categories as possible.

To grade the responses I used a thin wrapper around the well studied oKatanaaa/ifeval library (Kilbas, 2026), which itself is a clean wrapper around Google Research's own IFEval reference implementation, using the same verifier and evaluator class. Each row in the benchmark results CSV is graded by producing a verdict on whether it passes the strict or loose constraints. Strict requirements mean that the prompt response has to be exactly as defined by the grader, whereas loose requirements allow some room for error as long as it still maintains the essence the grader requires. A small adaptation strips None valued kwargs from the HuggingFace JSONL before they reach the verifier constructors. The main drawback here is inherent to IFEval itself, which is that heuristic verifiers can in principle be gamed to pass without actual instruction following (such as padding to hit a word count with filler words).

To make sure the evaluation is fair, the same 100 prompts were run across all three backends in both raw and RAG modes, producing 600 total prompt runs for analysis for this project. The single CSV schema mentioned in FR6 enables analysis from

this singular adapted grading script, with the grader noting the platform in the final graded CSV produced.

Chapter 5

Implementation

5.1 Backend abstraction

The backend layer allows any of the 3 backend classes to be called using a duck type contract, focusing specifically on the methods and attributes of the class rather than inheritance or a specific naming convention. This works well for a language like Python with runtime type checking instead of a language like C++ with compile time type checking. The backend definitions are shown in Listing 5.1. This allows the user to pass a –backend flag with the chosen model and the script will automatically use the chosen backend.

```
1 backends = {"llama_cpp": LlamaCppBackend, "rkllama":
     RkllamaBackend, "gemini": GeminiBackend}
2 backend = backends[args.backend](args)
```

Listing 5.1: Backends

The reason it is set up this way is that each backend's ___init___ method has vastly different initialisation costs (Loading a 4GB GGUF model to the GPU vs opening an HTTP request to the NPU / Cloud). A shared base class would either be empty or too awkward to make properly contractual. The return statement being shared is what truly matters, as every generate() method returns the same dictionary with all of the required measurements. The 22 column CSV is defined once at the top of the file as RESULT_FIELDS and every row is built from the same merger of args + res + e_stats.

The LlamaCppBackend class has a chance of yielding chunks that contain no visible output, so the TTFT is captured only on the first chunk containing non empty text after stripping the <think> tags. The <think> tag stripping is consistent across all 3 backends to allow for fair comparison.

```
for chunk in self.llm.create_completion(prompt=prompt,
    max_tokens=self.args.max_tokens,stream=True,temperature=
    self.args.temperature,):
        text = chunk["choices"][0]["text"]
        if "<think>" in text or "</think>" in text:
                text = text.replace("<think>", "").replace("</
                    think>", "")
                if not text:
                        continue
        if ttft is None and text:
                ttft = time.perf_counter() - t0
```

Listing 5.2: LlamaCppBackend

Qwen strips them by default when you don't ask for reasoning, and they inflate the output and sometimes waste all 1024 tokens on thinking without actually generating a response. It also helps prevent the IFEval grader from being confused by tag noise or irrelevant data.

A few other minor noteworthy parts of the Llama setup are as follows. It only loads the model once, as this takes a fair while - it saves a lot of time on the full benchmark. We also set the n_gpu_layers to 1 by default, as the 4GB Qwen3-4B Q4_K_M fits comfortably inside the RTX 4070's VRAM. Finally, we have small sanity checks on the reported TPS, in the form of "if decode_tps $>$ 500: decode_tps = n_output / total. Llama-cpp-python occasionally returns nonsense decode rates on short prompts, so we recalculate the decode_tps in this manner.

The RKllamaBackend is the most interesting part of the entire project, setup of which will be discussed in Section 5.5 The project runs it through the rkllama package, a

wrapper around RKLLM 1.2.3 that runs on the NPU itself and exposes a HTTP endpoint for the host script to pick up.

```
with requests.post(url, json=payload, stream=True, timeout
   =900) as r:
        r.raise_for_status()
        for line in r.iter_lines():
                if not line:
                        continue
        try:
                obj = json.loads(line)
        except json.JSONDecodeError:
                continue
        if obj.get("response"):
                if ttft is None:
                        ttft = time.perf_counter() - t0
                chunks.append(obj["response"])
        if obj.get("done"):
                last = obj
                break
```

Listing 5.3: RKllamaBackend

The last object is rkllama's final payload, which contains all the NPU device counters such as eval_count, eval_duration, etc.

5.2 Energy monitoring module

The energy monitoring setup is done with the TapoEnergyMonitor inheriting from the PynvmlEnergyMonitor, to keep the output to the CSV coherent regardless of the energy monitoring method chosen. Inheriting from the PynvmlEnergyMonitor was the clear choice as it'd already been developed and kept the restructuring cost low, only having to overwrite the ___init___ method.

The energy total is computed as shown in the following code snippet:

```
energy = sum(
        (self._samples[i] + self._samples[i - 1]) / 2.0 * (
           self._times[i] - self._times[i - 1])
        for i in range(1, n)
)
```

Listing 5.4: EnergyTotal

This is an implementation of trapezoidal integration (NumpyDocs, n.d.), where each pair of consecutive samples defines a trapezoidal area. This integration is standard as using a rectangular integration would lead to over or under estimating depending on whether anchored to the leading or trailing edge of the sample.

The Tapo Python SDK uses an async design, whereas the full energy monitor itself is synchronous as it lives in a polling thread. Bridging the two of these requires a unique event loop for each instance, with the implementation shown below. Slight foresight with ability to run the host script on other platforms was included, though ultimately irrelevant for this project in its current state.

```
        if sys.platform == "win32":
                self._asyncio_loop = asyncio.SelectorEventLoop
                   ()
        else:
                self._asyncio_loop = asyncio.new_event_loop()
```

Listing 5.5: Async bridge

We have to specify SelectorEventLoop() as Windows with Python 3.8+ defaults to ProactorEventLoop, which is more efficient but is incompatible with the Tapo SDK, returning empty data in initial testing before finding the underlying incompatibility.

The Tapo plug measures the whole system power at the wall socket, which makes it valid for the NPU and GPU. Section 6 reports full findings with the Tapo, interlinked with a smaller subset of data with pynvml only measurement for the GPU.

5.3 RAG pipeline

To build the Simple English Wikipedia corpus the benchmark.py host script has to be run with the single flag –build-rag-index as a one time operation. It streams from the wikimedia/wikipedia library and with the config as 20231101.simple from Hugging Face, truncates each article to 800 characters, and encodes batches of 500 at a time with all-MiniLM-L6-v2 encoding on the CPU. This results in a total of 464 shards totalling approximately 385MB.

```
def query(self, q: str, top_k: int = 2) -> str:
        if self.index is None or self.index.ntotal == 0:
                return ""
        embedder = self._get_embedder()
        q_emb = embedder.encode([q], convert_to_numpy=True)
        faiss.normalize_L2(q_emb)
        _, idxs = self.index.search(q_emb, top_k)
        passages = [f"- {self.all_docs[i][-600:]}" for i in
           idxs[0] if 0 <= i < len(self.all_docs)]
        return "\n".join(passages)
```

Listing 5.6: RAG retrieval

The retrieval uses IndexFlatIP, an inner product on L2 normalised vectors with cosine similarity. A key thing to note is that passages are tail truncated to the last 600 characters before being joined with the prompt. This is to keep the context window within bounds, as without it the 1024 token window could be fully utilised before even reaching the decode stage. The reason to choose the 600 character tail is that the end of articles on average contain denser information about a specific topic than the opening introduction. The join method on the prompt and the context is as follows:

```
        full_prompt = ptext
        if mode == "rag" and rag:
                ctx = rag.query(ptext, top_k=args.rag_top_k)
        if ctx:
```

```
                full_prompt = (
                        f"Use␣context␣to␣answer.␣If␣irrelevant␣,␣ignore.\n"
                        f"Context:\n{ctx}\n\nInstruction:␣{ptext}\nAnswer:"
        )
```

Listing 5.7: RAG Prompt

Every prompt is ran twice when the –rag-mode both is selected, and recorded as such in the CSV. Retrieval itself is not cached across runs, but as it is deterministic, every RAG prompt will re embed the same query for the same prompt, satisfying the NFR1 of reproducibility.

5.4 CSV handling

The CSV allows for resuming after interruption, as with the NPU the runs can take roughly 3 hours for 100 prompts, and if that failed halfway through, all the data would be redundant without a resume feature. In line with FR4, the run_benchmark scans the output CSV (if the –resume flag is set) and builds a set based on prompt_ids that already have a non empty response_text field.

```
        done = set()
        if args.resume and out_path.exists():
                try:
                        with open(out_path, "r", newline="", encoding="utf-8") as f:
                        for row in csv.DictReader(f):
                                if row.get("label") == args.label and row.get("response_text"):
                                        done.add((row.get("mode", ""), row.get("prompt_id", ""),
```

```
                                        int(row.get("
                                        run_idx") or 0)))
```

Listing 5.8: Resume functionality

The label filter here means that resuming an RAG focused run will not be confused with a raw run that shares prompt IDs. The response text non empty check means that a crashed or incomplete inference retries on resume, rather than being treated as completed.

Errors that occur in inference or throughout the process are logged into the CSV's notes field with the exception type, and then cleanly break and move onto the next prompt using try-catch blocks. An example of this is below.

```
1   try:
2           if monitor is not None:
3                   monitor.start()
4           try:
5                   res = backend.generate(full_prompt)
6                   if res.get("notes"):
7                           notes.append(res["notes"])
8           finally:
9                   if monitor is not None:
10                          try:
11                                  e_stats = monitor.stop
                                        ()
12                          except Exception as ex:
13                                  notes.append(f"
                                        monitor_stop_error:
                                        ␣{type(ex).__name__
                                        }")
14  except Exception as e:
15          notes.append(f"inference_error:␣{type(e).
              __name__}:{e}")
```

Listing 5.9: Inference error handling

The inner finally ensures that the monitor is stopped even if generate() raises an error, otherwise that polling thread will run forever leaking into the samples of the next prompt. The outer except catches anything that generate() raises after the monitor's cleanup.

In accordance with FR6 / NFR5, the 22 column CSV schema captures everything needed to analyse or grade the data without re running inference. This is what allows katanaifeval.py to be developed entirely separately from the benchmark.py script and output the relevant strict or loose gradings without ever touching a model.

5.5 RK3588 Setup

Setting up the RK3588 was an arduous process that is worth briefly documenting here. It is not possible to boot from USB with the Orange Pi 5 Plus, so I had to flash an SD card with the correct image and boot from that. Following that I wanted to utilise an M.2 NVMe I had acquired due to its major speed advantage and thus presumable inference speed increase relative to it. I utilised dd to move the operating system over, then changed the fstab UUID to boot from the NVMe.

Sometime in this process the SD card got stuck as a bootloader only, and while attempting to fix this I wiped a crucial part of the bootloading process and had to reflash. This will be discussed further in the conclusion as a limitation of the relatively new RK3588 hardware without wide community support, as I later discovered I may needed to reflash to obtain driver upgrades, but this wasn't possible with the remaining time of the project given the risks involved. Installing rkllama itself was as simple as creating a Python venv, git cloning the rkllama repository into it, then running rkllama-server to listen on port :8080.

Chapter 6

Results & Discussion

6.1 Results table

Below is the table of relevant results, with following sections explaining all of the notable data points from each specific backend, finishing with a cross platform comparison and finally an evaluation against the project's criteria.

The prefill TPS has to be recalculated as input_tokens/TTFT due to NPU measurement limitations

Platform	**InT**	**OutT**	**TTFT**	**Pre***	**Dec**	**J/tok**	**Pow**	**Str %**	**Lse %**
Gemini 2.5 Flash RAG	320	242	0.71	594	193.0	n/a	n/a	82.0	82.0
Gemini 2.5 Flash raw	46	416	0.56	86	171.8	n/a	n/a	82.0	86.0
RTX 4070 RAG	317	246	4.17	76	16.4	10.13	91.9	80.0	82.0
RTX 4070 raw	47	310	0.60	86	16.8	5.61	91.8	80.0	82.0
RK3588 RAG	327	263	5.83	56	6.3	1.83	8.7	77.0	81.0
RK3588 raw	57	287	3.41	16	6.5	1.55	8.6	78.0	80.0
*Prefill TPS recomputed as $input\_tokens/TTFT$									

Table 6.1: Summary of performance metrics by platform and mode

6.2 RTX 4070 results

All 200 prompts ran cleanly on the RTX 4070 with no crashes, and the tapo library successfully captured power readings for each row. Below are the means from the run, with the raw data first and the RAG data following:

TTFT: 0.60s / 4.17s

Decode: 16.75 TPS / 16.3 TPS

Prefill: 86 TPS / 76 TPS

J/token: 5.61 / 10.13

Wall socket average power: 92W flat across both

This close similarity between the two Decode TPS values, as shown in the figure below, proves that the RAG cost lives entirely in the prefill section. TTFT jumps 7x from 0.60s to 4.17s from raw to RAG because the input tokens climb massively from ~47 to ~317 due to prepending the RAG context. The J/token doubles from 5.61 to 10.13, again because of the RAG context prefill, but the key nuance here is that total energy per prompt is essentially the same.

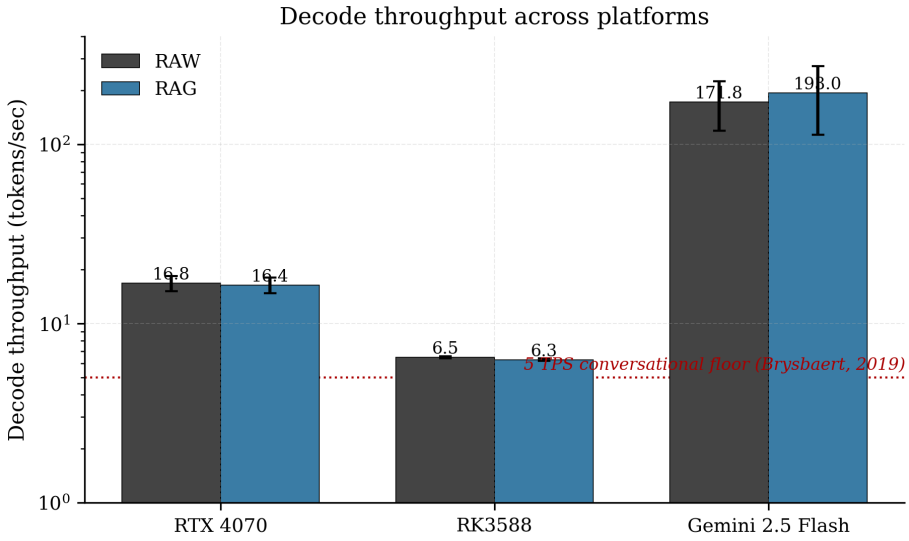

Figure 6.1: Decode throughput across platforms

Another noteworthy point is that despite the input tokens being 317 for RAG and

47 for raw, the output token count is smaller, suggesting that the longer the context, the more concise the output. This leads to J/token climbing, but total J/prompt actually staying the same. Per token is a great efficiency metric but total energy is also viable.

The IFEval pass rates are 80% for strict and 82% for loose in both modes, showing that RAG doesn't assist the 4070 in passing IFEval heuristics. This is expected because IFEval tests instruction following constraints, not factual recall which is where RAG shines. The Tapo power readings are remarkably stable across the entire run sitting at 91.8W average, ultimately making the J/token metric extremely reliable too.

6.3 Orange Pi 5 Plus NPU results

All 200 prompts ran cleanly on the RK3588 NPU too with no crashes, and the tapo library was also successful at recording the full power readings. Below are the averages for the run akin to the above 4070 section. As a quick sidenote, rkllama seems to report prefill TPS as values in the 10^8 range, so a workaround was applied to get it from input_tokens / ttft_s.

TTFT: 3.41s / 5.83s

Decode: 6.46 / 6.27 TPS

Prefill: 16 / 56 TPS

J/token: 1.55 / 1.83

Wall socket average power: ~8.65W

The total energy per prompt averages out to about ~425 J, roughly a quarter of the GPU's. The RAG Decode TPS number being higher seems strange at first, but the explanation is that the fixed per prompt overhead is spread over a much larger total input, meaning that the raw figure is overhead dominated. This is a slight methodology problem that should be revised in future work.

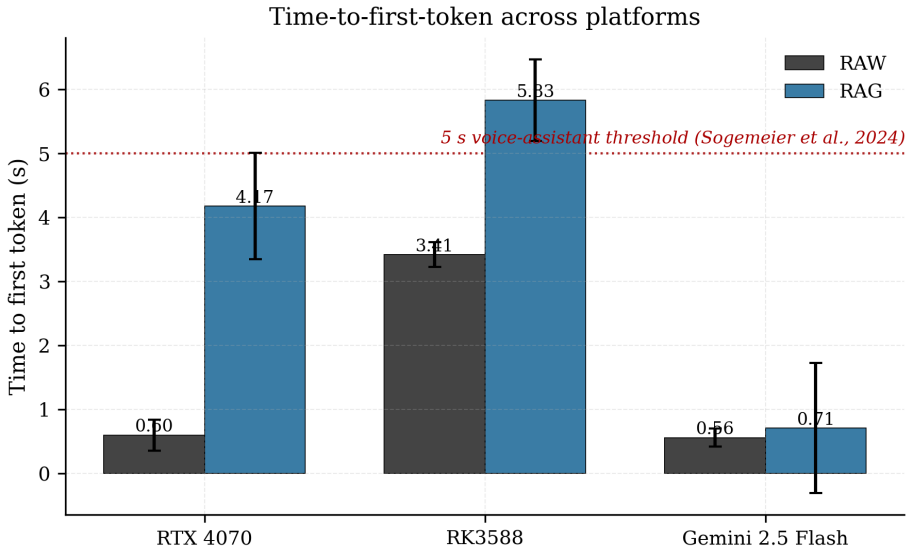

Figure 6.2: Time to first token across platforms

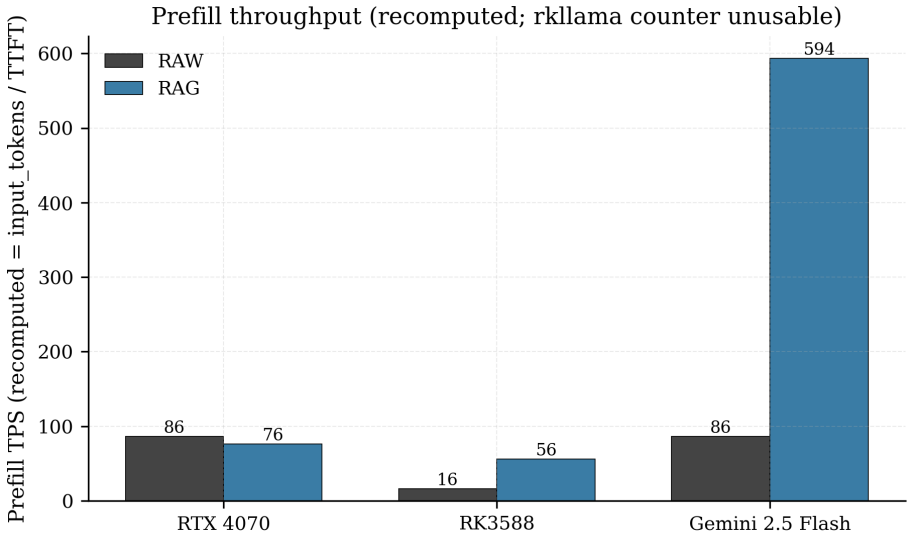

Figure 6.3: Prefill throughput

The Decode TPS of ~6.4 sits comfortably above the 5 TPS conversational floor derived from Brysbaert, 2019's work, with the TTFT in raw mode sitting comfortably under the 5s voice assistant threshold from Sogemeier et al., 2024's work. In the RAG mode it sits slightly higher but this is a comfortable trade off if RAG was only utilised on heavily factual prompts. All of this data combines to support the claim that **the RK3588 is currently viable for real time conversational AI**.

The driver mismatch also means that the reported numbers should be treated as the

floor of the RK3588's capability, this is conservative reporting that still proves a ~4x energy efficiency increase per token.

6.4 Cloud (Gemini) results

All 200 prompts ran successfully, though this was restricted by the free tier so the project upgraded to paid usage and used a whole £0.13 throughout the run. As mentioned before and seen in the table above the energy data is missing for Gemini, but the remaining measurements are outlined below

TTFT: 0.56s / 0.71s

Decode: 171.8 / 193.0 TPS

Prefill: 86 / 594 TPS

As expected, the Gemini output is in a class of its own, and generates output tokens at ridiculous speed. This is due to the raw throughput power of data centre scale hardware, and makes it a good quality anchor. The stand-out finding here is that the strict / loose pass rates between Gemini and the NPU are within a margin of ~5%, which is discussed further in the cross comparison in the following section.

Unfortunately there is no energy comparison possible here as mentioned in sections 2.2 and 4.5, as Google explicitly does not publish this data.

6.5 Cross platform discussion

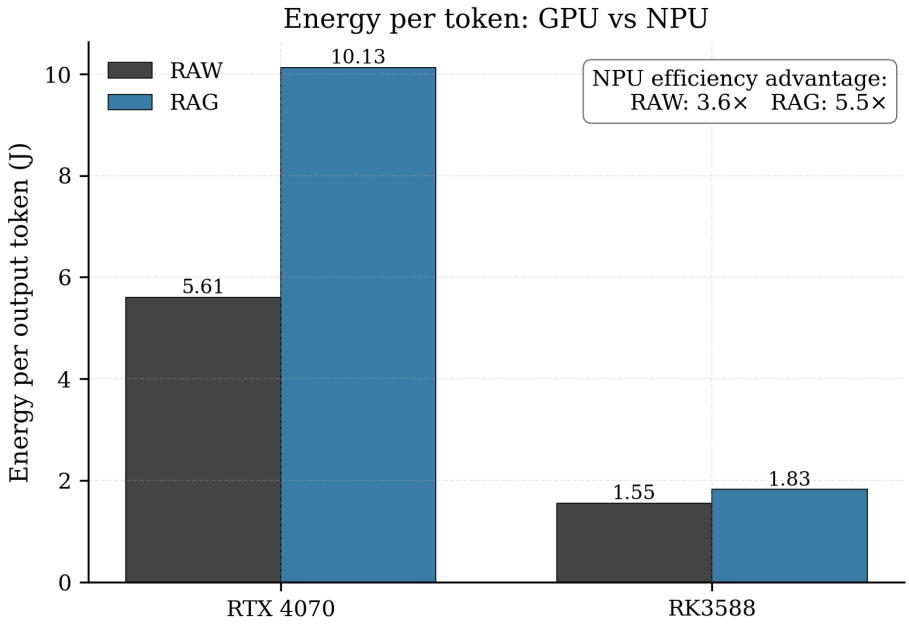

Figure 6.4: Energy per token

As shown above, the RK3588 NPU is 3.6x more efficient in raw mode, and 5.5x more efficient in RAG mode. The actual wall socket power difference is even larger at 10.6x (92W vs 8.7W). The J/token gap is smaller because the GPU completes each prompt faster, so while the GPU has a higher power draw it's offset by the shorter inference time. Both Tapo measurements are whole system, so the comparison is strong.

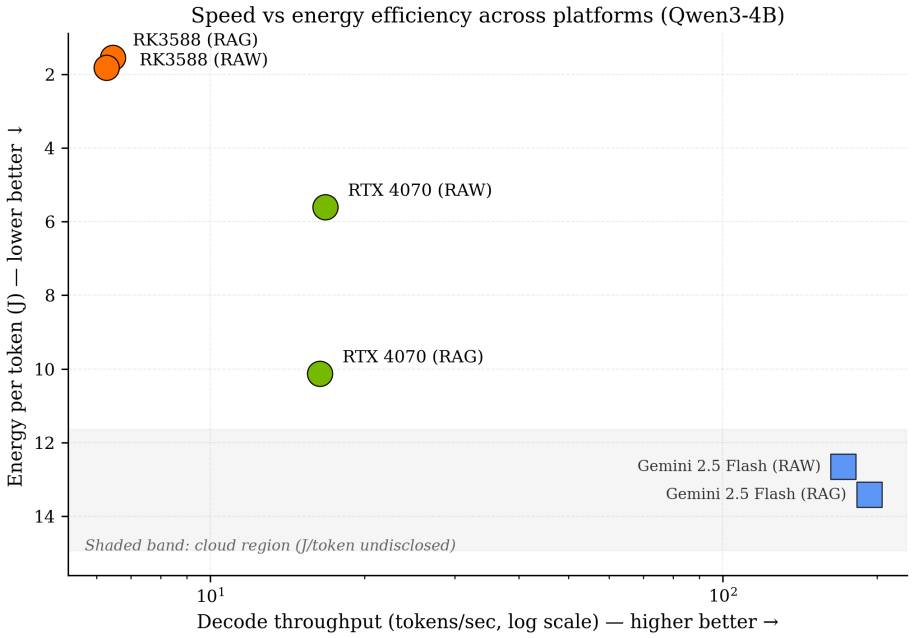

Figure 6.5: Speed vs efficiency scatter plot

The above figure shows the Decode TPS on the x-axis, and the J/token on the y-axis (inverted for readability), with 6 points total for the platforms and modes tested (3 platforms, RAG on or off). The shape of the graph tells us a lot. GPU sits in the middle, with high speed and high energy usage, the NPU sits in the low energy / moderate speed region, and Gemini sits at extremely high speed, but with energy usage. There is no single winner here, the graph shows that choice of platform is use case dependent. For interactive coding or summarisation, the GPU wins, but for always on assistants or energy conservative / privacy use cases, the RK3588 is clearly the winner. This is the central argument of the dissertation: power is wasted when a much simpler and more efficient AI can achieve the same usability as far more powerful (and wasteful) monitored alternatives.

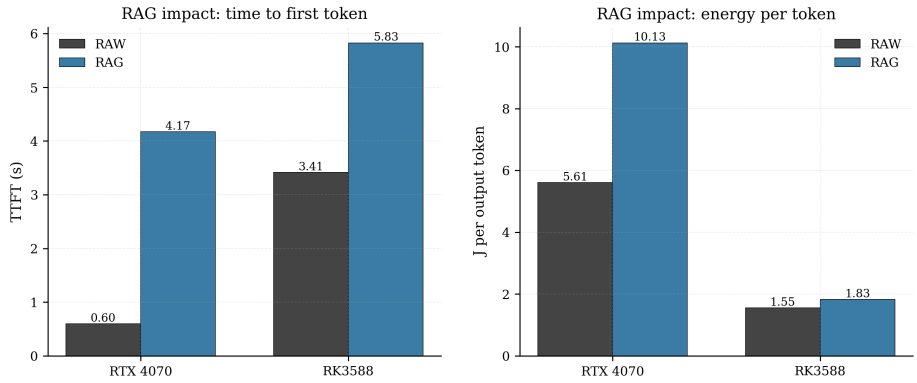

Figure 6.6: Impact of RAG

Both edge platforms pay a penalty in TTFT for RAG, however, the penalty is vastly different at 7x for the GPU and 1.7x for the NPU. Neither see much of a difference in decode TPS change. Energy per token scales differently too, at 1.8x for the GPU but only 1.18x for the NPU. This is because GPU power stays consistent during prefill, so extra input tokens are expensive in terms of energy, whereas the NPU prefill takes longer but at a much lower power cost. On the whole, RAG is relatively cheaper on the NPU than on the GPU, despite being slower in absolute time. This again leads to the conclusion that each of the platforms is more efficient in different areas depending on the use case / the optimisation target.

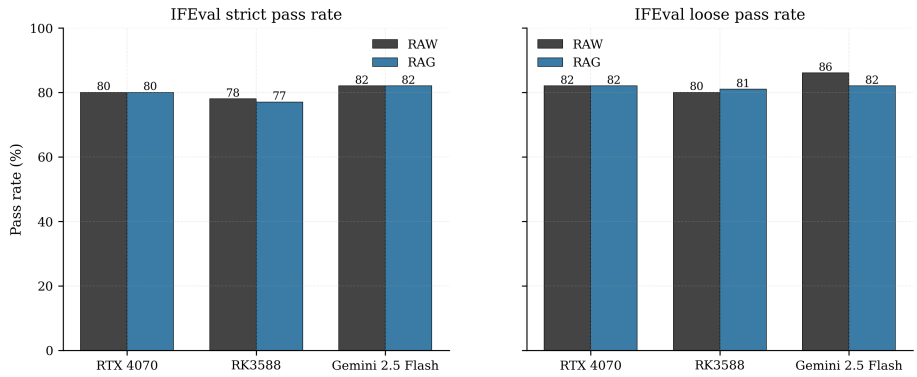

Figure 6.7: IFEval pass rates

Above are graphs showing the IFEval strict and loose pass rates. The gap between the £200 SBC and a frontier cloud model is only 4-5% points in strict and 4% loose. At similar parameter scales, the core instruction following quality seems to be dominated by the model or weights themselves, as opposed to the deployment

hardware, meaning that edge NPU AI is likely underutilised in modern AI use cases. Edge NPUs use a fraction of the energy, expose zero data to third parties, and achieve near equal results on the IFEval test.

6.6 Evaluation against criteria

In this section I will evaluate how successful the project was in meeting the objectives outlined in Section 1.2

Objective 1 (Cross platform benchmarking pipeline): Met, end to end deployment was achieved on a GPU, NPU and the cloud with a common schema, all driven from a single host script

Objective 2 (Energy per token measurement): Met for edge, not applicable for cloud. The wall socket measurement produced stable per prompt energy metrics.

Objective 3 (Instruction following assessment): Met using IFEval dataset for strict and loose pass rates with the oKatanaaa implementation.

Objective 4 (RAG implementation): Met using IndexFlatIP over the Simple English Wikipedia corpus, working on all 3 platforms. The raw vs RAG comparison was discussed and analysed, noting that RAG doesn't have a significant impact on IFEval pass rates, which itself is a meaningful result.

Objective 5 (Reproducible artefact): Met with a single python script with flags for options such as backend choices, modes and resume functionality, returning a single CSV schema across all platforms.

Chapter 7

Conclusion

7.1 Summary of achievements

This project built a cross platform LLM benchmarking artefact using an identical IFEval prompt set across an RTX 4070, RK3588 NPU and Gemini 2.5 Flash. A 22 column CSV schema allowed 1 to 1 comparison between all 3 platforms as a single source of truth. Energy monitoring used a TP-Link Tapo P110 smart plug to get wall socket energy readings for edge platforms. An IFEval grader assisted with the analysis of instruction following, and an RAG pipeline used FAISS with Simple English Wikipedia to benchmark RAG viability.

There are three concrete findings from this project. First, that the RK3588 NPU is 3.6-5.5x more energy efficient per token than the consumer RTX 4070 GPU, with the tradeoff being 2.6x slower decode, although still above the conversational floor. Second, the IFEval quality gap between the NPU, GPU and Cloud models is minor, small enough to justify edge deployment in privacy or energy efficient use cases. Third, RAG roughly doubles TTFT on both edge platforms, but gives no measurable improvement to IFEval, indicating that RAG is use case specific rather than holistically beneficial.

In my opinion and from the data in this project, edge AI is the only practical option for consumers prioritising energy efficiency and privacy in terms of inference. The dissertation provides the empirical basis needed to prove this and partially close the knowledge gap identified in Section 2.5.

7.2 Limitations

The RK3588 ran rknpu 0.9.6 against the rkllama recommended 0.9.7. The reported NPU figures are a floor, not a ceiling. The Tapo plug has a 1% accuracy deviation, which is adequate here but not ideal. Gemini energy is not measurable due to Google's lack of disclosure. No mobile class hardware was tested despite being a major competitor in the modern edge AI landscape in 2026.

Only a single model (Qwen3-4B) was tested across multiple platforms, the original plan included multiple models, but this was dropped for scope reasons. Quantization is slightly asymmetrical, with the GPU running at Q4_K_M against the NPU's Q8_0, so the comparison is not *strictly* identical. IFEval only tests the model's instruction following ability, not the factual accuracy, reasoning depth or open ended generation quality. No voice assistant pipeline was built to validate the conversational floor claims, although this has been heavily previously researched and referenced in the literature review.

7.3 Reflection on the development process

I planned a total of roughly two weeks to integrate the NPU, but it took closer to five weeks of consistent debugging and an almost bricked SD card to overcome it. The lesson I learned is that emerging hardware needs a 3x longer buffer time than the ideal path estimate. Future projects I undertake with similar hardware will account for this.

In future I would also build the CSV schema and source of truth on day one, not iteratively, as it was extremely valuable and could have been even more valuable if built first.

Moving from the initially planned Docker environment to a single host script saved a lot of time and allowed for a much more efficient comparison. I'm glad I didn't pivot to running the script on 3 different backends, this made the project possible.

I learned a lot during this project about time management and perseverance, and am honestly happy with the finished artefact and everything I learned along the way.

7.4 Future work

Reflashing the kernel to 6.1+ with the matching rknpu 0.9.7 driver and repeating the benchmark to discover how much performance is currently being left on the table would be viable future work. Adding Llama-3.1-8B and Phi-3-Mini as additional models would allow for much more model to hardware comparison. Adding a Raspberry Pi 5 was initially in the scope but had to be cut, it would be interesting to see how it fared against the NPU. Extending the grading to use LLM as a judge on prompt response quality could also yield interesting data. Adding multi turn conversational ability would allow data gathering on context degradation, alongside a possible voice assistant front end.

References

AmericanEnterpriseInstitute (2026). URL: https://www.aei.org/research-products/working-paper/chinas-transition-to-scalable-intelligence/ (cit. on p. 6).

Ammanath, Beena (2024). 'How to manage AI's energy demand—today, tomorrow and in the future'. In: *World Economic Forum*. Vol. 25 (cit. on p. 5).

Brysbaert, Marc (2019). 'How many words do we read per minute? A review and meta-analysis of reading rate'. In: *Journal of memory and language* 109, p. 104047 (cit. on pp. 8, 36).

Chen, Le et al. (Jan. 2025). *Characterizing Mobile SoC for Accelerating Heterogeneous LLM Inference*. en. URL: https://arxiv.org/abs/2501.14794v2 (cit. on p. 4).

Dettmers, Tim et al. (2022). 'Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale'. In: *Advances in neural information processing systems* 35, pp. 30318–30332 (cit. on p. 7).

Fan, Tianyu et al. (Jan. 2025). 'MiniRAG: Towards Extremely Simple Retrieval-Augmented Generation'. In: arXiv:2501.06713. arXiv:2501.06713. DOI: 10.48550/arXiv.2501.06713. URL: http://arxiv.org/abs/2501.06713 (cit. on p. 9).

Frantar, Elias et al. (2022). 'Gptq: Accurate post-training quantization for generative pre-trained transformers'. In: *arXiv preprint arXiv:2210.17323* (cit. on p. 7).

FTC (May 2023). en. URL: https://www.ftc.gov/news-events/news/press-releases/2023/05/ftc-doj-charge-amazon-violating-childrens-privacy-law-keeping-kids-alexa-voice-recordings-forever (cit. on p. 1).

HuggingFace (Apr. 2025). URL: https://huggingface.co/datasets/wikimedia/wikipedia (cit. on p. 3).

Kerr, Dara (July 2024). 'AI brings soaring emissions for Google and Microsoft, a major contributor to climate change'. en. In: *NPR*. URL: https://www.npr.org/2024/07/12/g-s1-9545/ai-brings-soaring-emissions-for-google-and-microsoft-a-major-contributor-to-climate-change (visited on 8th May 2026) (cit. on p. 4).

Kilbas, Igor (Apr. 2026). *oKatanaaa/ifeval*. Python. URL: https://github.com/oKatanaaa/ifeval (cit. on p. 23).

Kurt, Uygar (Jan. 2026). 'Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct'. In: arXiv:2601.14277. arXiv:2601.14277. DOI: 10.48550/arXiv.2601.14277. URL: http://arxiv.org/abs/2601.14277 (cit. on p. 7).

Kwon, Woosuk et al. (2023). 'Efficient memory management for large language model serving with pagedattention'. In: *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626 (cit. on pp. 6, 8, 20).

Li, Pengfei et al. (2025). 'Making ai less' thirsty''. In: *Communications of the ACM* 68.7, p. 1 (cit. on p. 1).

Lin, Ji et al. (2024). 'Awq: Activation-aware weight quantization for on-device llm compression and acceleration'. In: *Proceedings of machine learning and systems* 6, pp. 87–100 (cit. on p. 7).

NumpyDocs (n.d.). URL: https://numpy.org/doc/stable/reference/generated/numpy.trapezoid.html (cit. on p. 28).

Oviedo, Felipe et al. (2025). 'Energy Use of AI Inference: Efficiency Pathways and Test-Time Compute'. In: arXiv:2509.20241. arXiv:2509.20241. DOI: 10.48550/arXiv.2509.20241. URL: http://arxiv.org/abs/2509.20241 (cit. on p. 2).

Patterson, David et al. (2022). 'The carbon footprint of machine learning training will plateau, then shrink'. In: *Computer* 55.7, pp. 18–28 (cit. on p. 4).

Reimers, Nils and Iryna Gurevych (Aug. 2019). 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks'. In: arXiv:1908.10084. DOI: 10.48550/arXiv.1908.10084. URL: http://arxiv.org/abs/1908.10084 (cit. on p. 22).

Schwartz, Roy et al. (Nov. 2020). 'Green AI'. en. In: *Communications of the ACM* 63.12, pp. 54–63. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3381831. URL: https://dl.acm.org/doi/10.1145/3381831 (cit. on p. 5).

Shi, Tianyao and Yi Ding (Aug. 2025). 'Systematic Characterization of LLM Quantization: A Performance, Energy, and Quality Perspective'. In: arXiv:2508.16712. arXiv:2508.16712. DOI: 10.48550/arXiv.2508.16712. URL: http://arxiv.org/abs/2508.16712 (cit. on p. 7).

Sogemeier, Denise et al. (Sept. 2024). 'The Importance of Timing—An Expert Evaluation on Latencies for Voice Assistants'. en. In: *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* 68.1, pp. 1098–1100. ISSN: 1071-1813, 2169-5067. DOI: 10.1177/10711813241260290. URL: https://journals.sagepub.com/doi/10.1177/10711813241260290 (cit. on pp. 8, 36).

Strubell, Emma, Ananya Ganesh and Andrew McCallum (2019). 'Energy and policy considerations for deep learning in NLP'. In: *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 3645–3650 (cit. on p. 4).

Subramanian, Shreyas, Vikram Elango and Mecit Gungor (2025). 'Small language models (slms) can still pack a punch: A survey'. In: *arXiv preprint arXiv:2501.05465* (cit. on pp. 8, 21).

U.S. Energy Information Administration (2024). *U.S. Energy Facts Explained*. URL: https://www.eia.gov/energyexplained/us-energy-facts/ (cit. on p. 1).

Zhou, Jeffrey et al. (Nov. 2023). *Instruction-Following Evaluation for Large Language Models*. en. URL: https://arxiv.org/abs/2311.07911v1 (cit. on p. 3).